



Hello everyone, welcome to this exciting course. My name is Pablo Farias and I will be your guide throughout this learning journey. In this course, we will be developing a language learning game using JavaScript. This is a project-based course where we will tackle every single aspect of this application, from creating technical requirements to implementing its logic and functionality.

Course Prerequisites

This course assumes that you have a basic familiarity with JavaScript. In particular, you should have been exposed to concepts such as:

- Variables
- Functions
- Objects and Methods
- Conditional Statements
- Loops
- DOM API

It is important to mention that this course is beginner-oriented, so do not worry if you are new to programming or game development. All concepts and techniques will be explained in a clear and easy-to-understand manner.

About Zenva

Zenva is an online learning academy with hundreds of thousands of students. We offer a wide range of courses for people who are just starting out or for those who want to learn something new. Our courses are versatile, allowing you to learn in the way that suits you best.

- View the video tutorials
- Read the lesson summaries
- Follow along with the instructor with the included project files

Thank you for choosing to learn with us. We are looking forward to getting started and seeing you in the next lesson.

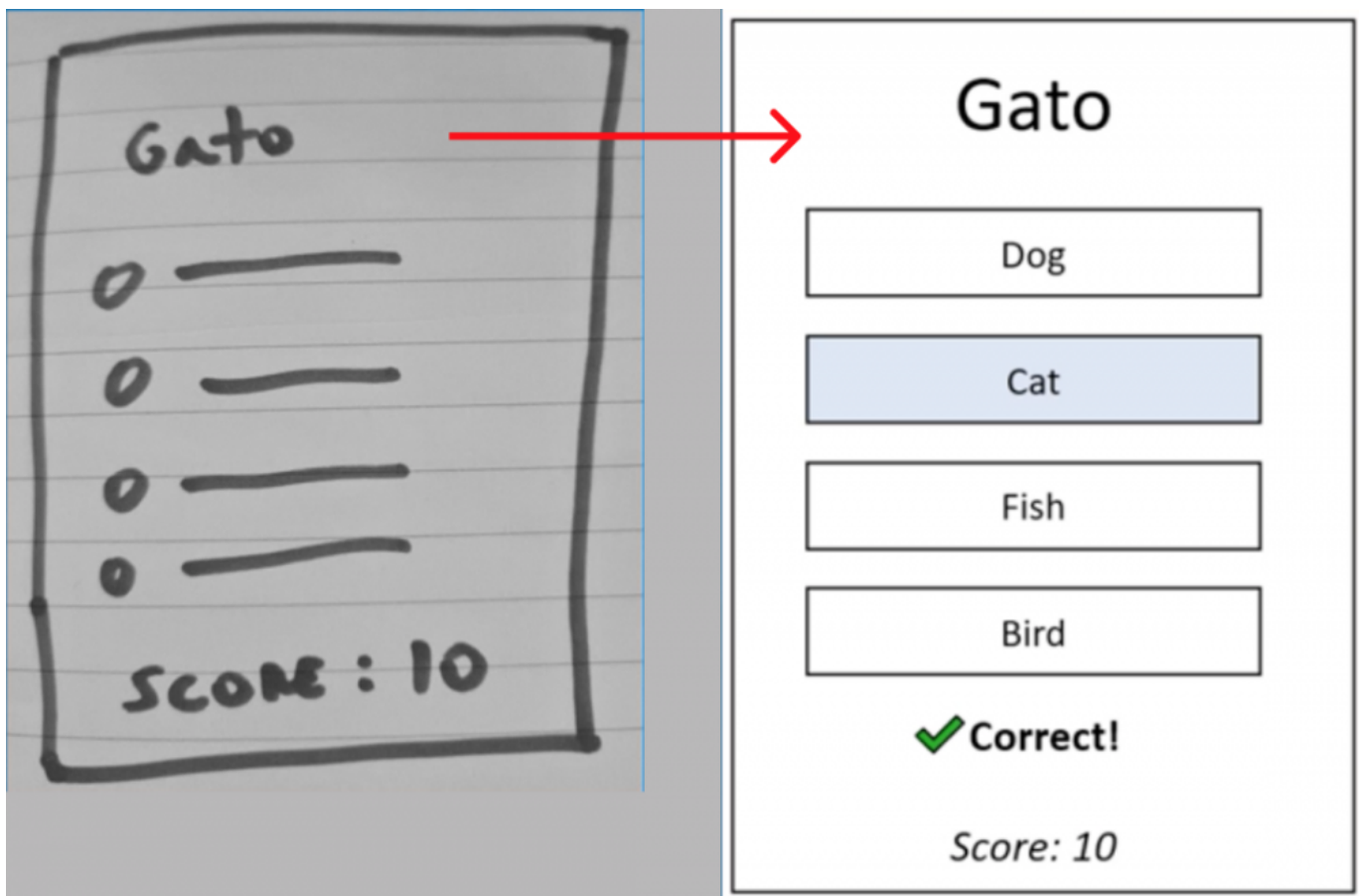
Welcome to this project-based course where we'll develop all aspects of a language learning app in JavaScript including creating technical requirements, implementing logic and functionality, and styling our app. In this first lesson, we'll review the app we'll be building as well as some of the thought process behind it. Let's get started!

Research Alternatives

Whether starting a new project or creating a new feature of an existing app it can be good to look at websites or apps that already exists with similar goals as the app you are building. This can give you an idea of what your users might expect, as well as find opportunities not yet covered. In the case of a language learning app, reviewing popular apps such as Duolingo or Rosetta Stone will give you a feel for what is already out there.

Mockup

Next, we'll start with a very rough draft of what we want to build. This is often called a **mockup**. This low-fi mockup could be sketched in a drawing app or even on paper. The key is **not** to be detailed. Instead, our focus is on layout and flow which provides a good starting point for thinking about or discussing ideas. From this rough draft we can gradually develop more precise mockups as our app design becomes more clear to us. The more precise mockups provide greater specifics on how you will develop your solution.



Technical Requirements

Once we have a **mockup** we then define **technical requirements** for our website or app which is a more precise description of what we will be building. These requirements include details such as:

**Technical Requirement****Description**

Platform: WEB

What platform(s) are we designing our app to run on

App Logic (data, flow, user interaction)

What data will our app need to use/store including local and remote storage? How will the app flow from one page/screen/scene to the next based on things like correct answer, incorrect answer, or game over? How does the user interact with our app?

Look and Feel

Once the app structure, data, flow and logic are determined, we then concentrate on how our app will look and feel to the end user

App Flow

Let's look at the **flow** in our language learning app. Based on our high-level **mockup** and **technical requirements** we take our app design to the next level of detail. We will start by showing the user a word in Spanish, followed by possible alternatives in English. The user picks an option and our app responds to indicate the result. Next, we update the score and either show the next question, start over or end the game.



App Data

Next, let's consider what data this app will need. This includes data that needs to be present before the app is opened and also data that will be generated as the app is used. It's important to note that there are generally more than one solution. Also, it's okay if you don't know all the answers up front. Data design, like other types of design, are based on requirements and assumptions which may change as the app is developed. The important thing is to think through the data needed, where is it going to be stored, how is it going to be transmitted, when is it needed, as well as how it will be used. An initial list of data may look similar to the following:

Data Requirements

Question Bank including Spanish words, English alternatives to show as well as the correct answer

Order of Questions

Current position in the questions list



Result of user selection

User Score

App Development Plan

It is common to develop an app in iterations, that is, we start with the most basic app architecture and progressively enhance the app's features. For our app, we will start with a simple architecture and get it working. We will then add game logic and additional features before turning our focus to the app's look and feel. The table below outlines our Game Plan:

Game Plan

First, start by showing one question on the screen

Second, enable user input and answering logic

Third, improve the structure of our app

Fourth, extend app to handle multiple questions

Fifth, add score and user feedback to our app

Lastly, once the logic is complete we will add CSS style to enhance app look and feel

Let's start coding in the next lesson!



In this lesson we will start by simply showing a single question on the web page. To do this we will represent the question in a JavaScript variable, then display the question in HTML.

Gato

Dog

Cat

Fish

Bird

Project Setup

Like any web development project we need two key tools: code editor and browser. You can use your favorite code editor or an online development environment together with Google Chrome, Firefox or any modern web browser. In this course we use the free and popular Visual Studio Code (<https://code.visualstudio.com/>). In your code editor, create two files.

Code File	Description
index.html	Our website HTML code for structuring and displaying our app in a web page
script.js	Our JavaScript code for app logic and functionality

Question - HTML Web Page

Let's starting by setting up the structure of our web page to display our **question** and **alternatives**. Inside **index.html** add the following HTML **DOM** (Document Object Model) **elements** as well as a reference to our JavaScript code file, **script.js**

```
<div id="title"></div>
<ul>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
</ul>

<script src="script.js"></script>
```

HTML Code	Description
<div id="title"></div>	The <div> tag is a generic HTML element. We provide an id so we can uniquely reference this HTML element from within our JavaScript code and CSS styling
	An Unordered List (tag) is a HTML element used to wrap HTML List Item elements (tags)
<li class="alternative">	Add four List Items (tags) to hold the four alternatives we show the user to select from. We provide each List Item with a CSS class of alternative so we can reference them from within our JavaScript code and CSS styling
<script src="script.js"></script>	At the bottom of our index.html file we add a reference to our JavaScript code file , named script.js , which will be loaded by the browser when this HTML web page loads

Question - JavaScript Object

Let's start coding inside **script.js**, setting up our **question** as a JavaScript **Object**.

```
// define the object for the question entity
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
```



```
correctAnswer: 1
};
```

JavaScript Code

```
let question = { properties: values };
```

```
title: 'gato',
alternatives: ['dog', 'cat', 'bird', 'fish'],
correctAnswer: 1
```

Description

Using the JavaScript **let** keyword and the curly braces (**{...}**), we set the variable **question** to be equal to a **JavaScript Object** which can contain any number of **property:value** combinations. To complete the syntax, follow the curly braces with a **colon**

Using the JavaScript Object **Property:Value** syntax (**property** and **value** separated by a **colon** and each combination separated by a **comma**). Here we assign our **question** variable three properties: **title** (String), **alternatives** (Array) and the **array index** of the **correctAnswer** (Number)

Question - showQuestion() Function

To keep our code organized and flexible we will create a JavaScript **Function** to show the question (**showQuestion**). This will enable us to execute the function by calling it, passing in the **question** object.

```
// define the object for the question entity
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};
```

```
// function for showing the question
function showQuestion(q) {
  // code
}
```

```
// call the function
showQuestion(question);
```

JavaScript Code

```
function showQuestion(q) { // function code }
```

```
showQuestion(question);
```

Description

Function Definition. We define a function named **showQuestion** and set it up to **receive** a **parameter** named **q**, which will represent the question inside the function block

Execute Function. We call the function named **showQuestion** passing it the **question** object previously defined as an **argument**

Working with the DOM

The **DOM** connects the **web page** to our **JavaScript** code. Inside **script.js**, we use DOM Methods on the **Document** (Object representing the currently loaded web page) to select DOM Elements.

Working with the DOM from JavaScript is a two step process. First, we **select** the DOM Element then we **modify** the DOM element.



Question - Show on Web Page

In our case, the **question word** (Spanish word) is stored in the **question Object** under a **String** property called **title**. In the HTML we represented the question word as a **<div>** with an **id** of **title**. Inside the **showQuestion** function, add logic to **select** and **modify** this DOM element.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// modified code
function showQuestion(q) {
  // new code
  let titleDiv = document.getElementById('title');
  titleDiv.textContent = q.title;
}

// existing code
showQuestion(question);
```

JavaScript Code

```
let titleDiv = document.getElementById('title');
```

```
titleDiv.textContent = q.title;
```

Description

Using the DOM method **getElementById()**, passing it the **id** as a **String** (in our **index.html** file we set this **id** to be **title**), we **select** the element storing a reference to it in a variable (**titleDiv**)

With a reference to the DOM element stored in a variable (previous step), we can now **modify** it. To do this, we set the **textContent** property of the DOM element (**<div>** with an **id** of **title**) to be the **title** property stored in our **question** variable (received as parameter **q** inside our function)

The **web page** now shows our **question** followed by an **empty Unordered List** (represented by default with bullet points) which will hold our **alternatives**.

gato



Alternatives - Show on Web Page

In our case, the **alternatives** (English words) are stored in a **question Object** under an **Array** property called **alternatives**. In the HTML we represented the alternatives as an Unordered List (****) containing List Items (**** with a **class** of **alternative**). Inside the **showQuestion** function, add logic to **select** and **modify** these DOM elements.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// modified code
function showQuestion(q) {
  // existing code
  let titleDiv = document.getElementById('title');
  titleDiv.textContent = q.title;

  // new code
  let alts = document.querySelectorAll('.alternative');
  console.log(alts);
  alts.forEach(function(element, index){
    element.textContent = q.alternatives[index];
  });
}

// existing code
showQuestion(question);
```

JavaScript Code

```
let alts =
document.querySelectorAll('.alternative');
```

Description

Since we are selecting DOM elements by **class** (multiple DOM elements returned) rather than by **id** (one unique element returned) as we did above, we use the DOM method



```
console.log(alts);
```

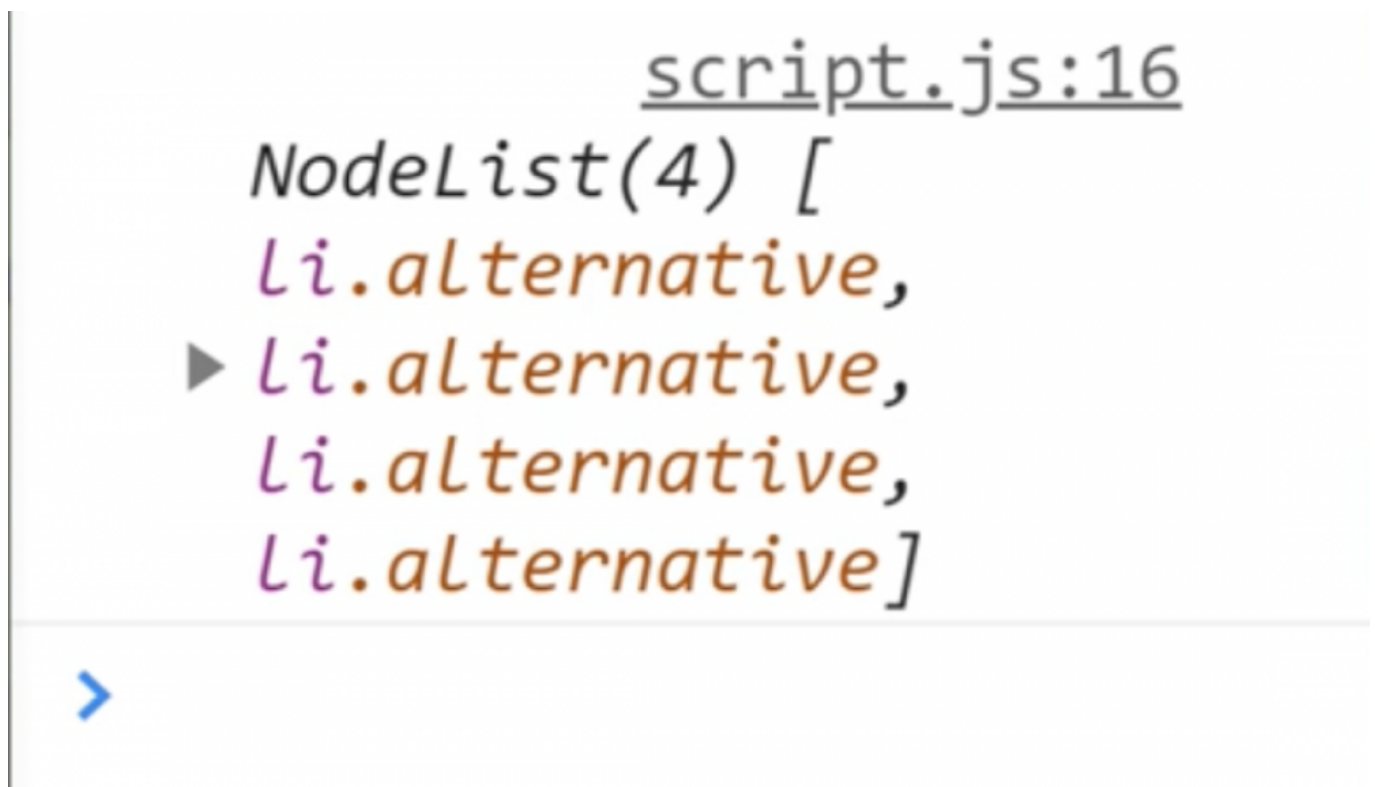
```
alts.forEach(function(element, index) {  
  element.textContent = q.alternatives[index];  
});
```

querySelectorAll(), passing it a **CSS Selector** (in this case, **class name** preceded by a period - **‘.alternative’**). A list of matching elements is returned and we store a reference to these elements in a variable (**alts**)

Log the variable **alts** to the **console** (see image below). Notice that **alts** contains each List Item element (**** with **class of alternative**) in an array-like ordered list called a **NodeList**

We can now **modify** each element to contain one element from our **question** variable **alternatives** array property. To do this, we set the **textContent** property using the JavaScript **forEach** method which loops over each **element** in the **alts** variable

Console shows a list of DOM elements which will contain our question **alternatives** (**** with **class of alternative**).



The **web page** now shows our **question** followed by our list of **alternatives**.



gato

- dog
- cat
- bird
- fish

Live Coding - Showing a Question

How can only one question be shown on the screen.



In this lesson we introduce the concept of **handling user input**. We will do this by adding an event listener to the **alternatives** List Items so we can check if the user picks the correct answer.

Event Listeners - Simple Button Click

Before we add the functionality for our user to choose the answer they believe is correct, let's start with a simple example of using an **event listener** to know when a user has **clicked a button**. When the user clicks the button, we log the message **Clicked!** to the **console**. Starting with the code from the last lesson, add the following code **below** the **showQuestion(question)** function call.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// existing code
function showQuestion(q) {
}

// existing code
showQuestion(question);

// new code
let btn = document.getElementById('btn');
btn.addEventListener('click', function() {
  console.log('Clicked!');
});
```

JavaScript Code

```
let btn = document.getElementById('btn');
```

```
btn.addEventListener('click', function() {
  console.log('Clicked!'); });
```

Description

Select Button Element. Using the DOM method **getElementById()**, passing it the **id** as a **String** (in our **index.html** file we set this **id** to be **btn**), we **select** the Button element storing a reference to it in a variable (**btn**)

Add Click Event Handler. With a reference to the Button element (**btn**) which is known as the **Event Target**, we can now attach an **event handler** by calling the method **addEventListener()**, passing it a function with two arguments: a case-sensitive String representing the **event type** we are listening for (**'click'**) and a **function** to call when the **event occurs**. When the event occurs, we run the code inside the function which in this case logs the word **Clicked!** to the console

gato

- dog
- cat
- bird
- fish

Click Me

Console

```
> Clicked!  
✓
```



Note: you can **optionally** remove the code we just wrote as an example before continuing with the next section.

Event Listeners - Alternatives List

Next, let's add **event listeners** to each List Item element (`<li>` with a **class** of **alternative**). As we did in the above Button example, we could assign each element an **id**, then select and add the event listeners one by one. However, since we already have a reference to this list from our last lesson (stored in the variable **alts**), we can add the **event listeners** to each List Item element (`<li>` with a **class** of **alternative**) inside the existing **alts.forEach()** method. Recall that the function passed to the **alts.forEach()** method has two parameters, **element** and **index**. We can use the **element** parameter (each **alternative**) as our **Event Target** to attach the **event handler** to on each iteration of **forEach**. We can then use the second parameter, **index**, to check against our question's **correctAnswer** property. If this comparison is **true** then we log **Correct Answer!** to the console else we log **Wrong Answer!**

```
// existing code  
let question = {  
  title: 'gato',  
  alternatives: ['dog', 'cat', 'bird', 'fish'],  
  correctAnswer: 1  
};
```

```
// existing code
```



```
function showQuestion(q) {
  // existing code
  let titleDiv = document.getElementById("title");
  titleDiv.textContent = q.title;

  // existing code
  let alts = document.querySelectorAll('.alternative');

  // modified code
  alts.forEach(function(element, index){
    // existing code
    element.textContent = q.alternatives[index];
    // new code
    element.addEventListener('click', function(){
      if (q.correctAnswer == index) {
        console.log('Correct Answer!');
      } else {
        console.log('Wrong Answer!');
      }
    });
  });
}

showQuestion(question);
```

Click each **alternative** one at a time and notice the **result** in the **console**. The only one that returns **true** is in fact the **correct answer!** (**gato** is Spanish for **cat**).



gato

- dog 1
- cat 2
- bird 3
- fish 4

Console

```
>
Wrong Answer 1
Correct Answer 2
Wrong Answer 3
Wrong Answer 4
> █
```



In this lesson we begin to modify our app's **architecture**. In the first few lessons we added and validated some basic functionality. The problem with the current architecture is that it is only really valid for asking one question. We could, of course, call **showQuestion(question)** multiple times but that will lead to multiple event listeners and other pain points as our app grows.

Refactor - Event Listeners

The first step is to find a way of only adding event listeners to our question **alternatives** only **once** (when the app first starts). To do this we refactor our code by creating a **start** function and moving our **event logic** inside it. Then this **start** method can kick off our functionality by showing the first question.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

function start() {
  // add event listeners

  // show first question
}

function showQuestion(q) {
  // show question title

  // show alternatives
}

start();
```

This **new architecture** is the next iteration in our app design so we expect that it will continue to evolve. Our current **refactoring goal** is to **separate** the **startup** code from the **show question** functionality. We do this by creating a **start** function which will **add event listeners** and then **show the first question**. Next, we reduce the responsibility of the **showQuestion** function to simply show the **question** and **alternatives**. To kick off our refactored code, call the **start** function as the last line of code.

Refactor - Start Function

With an overview of our refactored code presented in the previous section, let's focus on refactoring our event logic into the new **start** function. Once event handlers have been put in place, we add the code to show the first question by calling **showQuestion(question)**.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

function start() {
  // get alternatives
```



```
let alts = document.querySelectorAll('.alternative');
// add event handlers
alts.forEach(function(element, index){
    element.addEventListener('click', function(){
        // check correct answer
        console.log('check correct answer');
    });
});

// show first question
showQuestion(question);
}

function showQuestion(q) {
    // show question title

    // show alternatives
}

start();
```

Within the **start** function, we select all the **alternatives** (`<li>` with a class of **alternative**). This can be copied (exactly) from our previous **showQuestion** function. Next, copy the **alts.forEach** method and remove all the code inside its function. For now, we simply put a placeholder which logs to the console the text **check correct answer**. Since the **start** function will only be called when the app starts, we have succeeded in our goal of adding event listeners to our question **alternatives** only **once** (when the app first starts). Finally, placing the call to the **showQuestion** function at the bottom of the **start** function ensures our user experience begins as expected.

Refactor - showQuestion Function

Next, let's focus on **refactoring** the **showQuestion** function to reduce its responsibility to simply **show** the **question** and the **alternatives**. To do this, we **remove** all **code** associated with adding **event handling**.

```
// existing code
let question = {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
};

// existing code
function start() {
    // set event handlers

    // show first question
    showQuestion(question);
}

// modified code
function showQuestion(q) {
    // show question title
```



```
let titleDiv = document.getElementById('title');
titleDiv.textContent = q.title;

// show alternatives
let alts = document.querySelectorAll('.alternative');
alts.forEach(function(element, index){
  element.textContent = q.alternatives[index];
});
}

start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

New Code Flow	Description
let question = { properties:values }	Define question variable and assign it an Object containing properties and values for title (String), alternatives (Array) and correctAnswer (Number)
start();	Execute the start function passing no arguments. This kicks off our app, adds event handlers, and shows the first question
showQuestion(question);	Execute the showQuestion function, as the last step within the start function, to show the question and its alternatives on the web page
let question = { title: 'gato', alternatives: ['dog', 'cat', 'bird', 'fish'], correctAnswer: 1 };	
function start() { // get alternatives let alts = document.querySelectorAll('.alternative'); // add event handlers alts.forEach(function(element, index){ element.addEventListener('click', function(){ // check correct answer console.log('check correct answer'); }); }); // show first question showQuestion(question); }	
function showQuestion(q) { // show question title	



```
let titleDiv = document.getElementById('title');
titleDiv.textContent = q.title;

// show alternatives
let alts = document.querySelectorAll('.alternative');
alts.forEach(function(element, index){
  element.textContent = q.alternatives[index];
});
}

start();
```

Notice that the **web page** displays as before. For now, with placeholder code, each time we **click** an **alternative** the text **check correct answer** appears in the console.

gato

- dog 1
- cat 2
- bird 3
- fish 4

Console

```
>
check correct answer 1
check correct answer 2
check correct answer 3
check correct answer 4
> █
```



In this lesson we continue to evolve our app's **architecture**. It's important to note that there are many schools of thought on how best to structure an application. What we are showing in this course is one way design and evolve your app. We started with simply getting our basic functionality working. We then refactored our code to **separate** the activities involved in **starting our app** (one-time event) from the activities involved in **displaying a question** and its **alternatives** on the **web page**. In this lesson we continue to refactor our code by representing our app as a JavaScript Object which can have properties to keep track of data and methods for functionality.

Refactor - Create App Object

We will start by creating an **app** Object, then one by one convert our **independent functions** to **app methods**. We know our **app** will have two methods: **start** and **showQuestion**. We create these methods by setting a **property** name with a **value** equal to a **function**. Be sure to separate each property and method of an Object with a **comma**. Also, notice the modification to executing the **start** function now that it's a method of the **app** Object (**start();** becomes **app.start();**)

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// new code
let app = {
  start: function () {

    // get alternatives

    // show first question
  },

  showQuestion: function (q) {
    // show question title

    // show alternatives
  }
};

// modified code
app.start();
```

Refactor - start Method

To refactor the **start** function to **app.start** method requires two steps. First simply copy all the code from within the **start** function and paste it inside the **start method** function block of the **app** Object. Second, similar to how we changed **start()** to **app.start()** in the previous section; we need to change the execution of the **showQuestion** function to call the **showQuestion method** of the **app** Object. Since both methods, where we are calling from and where we are calling to, are inside the **app** Object we need to use the JavaScript **context** keyword **this** in front of any property or method reference. Therefore, **showQuestion(question);** becomes **this.showQuestion(question);**



```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// modified code
let app = {
  start: function () {
    // copied code

    // get alternatives
    let alts = document.querySelectorAll('.alternative');
    // add event handlers
    alts.forEach(function (element, index) {
      element.addEventListener('click', function () {
        // check correct answer
        console.log('check correct answer');
      });
    });

    // modified code (add keyword: this)

    // show first question
    this.showQuestion(question);
  },

  showQuestion: function (q) {
    // show question title

    // show alternatives
  }
};

// existing code
app.start();
```

Refactor - showQuestion Method

To refactor the **showQuestion** function to **app.showQuestion** method simply copy all the code from within the **showQuestion** function block and paste it inside the **showQuestion method** function block of the **app** Object.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

// modified code
let app = {
```



```
start: function () {
  // existing code

  // get alternatives
  let alts = document.querySelectorAll('.alternative');
  // add event handlers
  alts.forEach(function (element, index) {
    element.addEventListener('click', function () {
      // check correct answer
      console.log('check correct answer');
    });
  });

  // show first question
  this.showQuestion(question);
},

showQuestion: function (q) {
  // copied code

  // show question title
  let titleDiv = document.getElementById('title');
  titleDiv.textContent = q.title;

  // show alternatives
  let alts = document.querySelectorAll('.alternative');
  alts.forEach(function (element, index) {
    element.textContent = q.alternatives[index];
  });
}
};

// existing code
app.start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

New Code Flow	Description
let question = { properties:values }	No Change. Define question variable and assign it an Object containing properties and values for title (String), alternatives (Array) and correctAnswer (Number)
app.start();	Change since start is now a method of app . Execute the app.start method passing no arguments. This kicks off our app, adds event handlers, and shows the first question
this.showQuestion(question);	Change since showQuestion is now a method of app therefore we must use the JavaScript context keyword this to reference its parent app Object. Execute the showQuestion



function, as the last step within the **app.start** method, to show the **question** and its **alternatives** on the web page

Note: remove any **code** that is no longer being used such as the original **start** and **showQuestion** function blocks.

gato

- dog 1
- cat 2
- bird 3
- fish 4

Console

```
>  
check correct answer 1  
check correct answer 2  
check correct answer 3  
check correct answer 4  
> █
```



In the last lesson we refactored our app to be contained in a JavaScript Object which can hold together related **properties** such as data and functionality (referred to as **methods**). In this lesson, we continue to develop our **app** Object by adding a new method, **checkAnswer**, to check if the user selected the correct answer. We will also consider a very important and confusing point in JavaScript which is ensuring we have the right **context** (which Object the property or method belongs to) represented by the JavaScript keyword **this**.

The Value of THIS in JavaScript

In most cases, the value of **this** is determined by **how a function is called** (runtime binding). To better deal with the keyword **this**, modern JavaScript has two ways to ensure its value. One option is attach the **bind()** method to a **Function Definition** passing it the value of **this**, for example: **function() { /*code */ }.bind(this)**. The second option is to use JavaScript **Arrow Function** syntax instead of a regular Function Definition. This is especially powerful when passing functions as arguments as we do in our **start** method for **adding event listeners** and again for **handling the user clicks** inside that event handler because the **this** context is handled for you by JavaScript.

Refactor Checking Answers

We currently handle checking for correct answers inside the **start** method (run only once at app start). While this is a good place to set event listeners (done once), it is not a good place for logic that will be run multiple times throughout the app such as checking for correct answers. Let's start by adding a new method to our **app** Object called **checkAnswer**. Remember to separate this new method from the other methods inside the **app** Object with a **comma**.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {
  // existing code
  start: function() {
    // set event handlers
    // check answers
    // show first question
  },

  // existing code
  showQuestion: function(q) {
    // show question title
    // show alternatives
  },

  // new code
  checkAnswer: function(userSelected) {
  }
};

app.start();
```

Refactor START Method with Bind()



With the new method, **checkAnswer**, added to our **app** Object we can refactor the **start** method to move the check answers logic to this new method. To begin, we simply **replace** the placeholder **console log** with a **call** to our **new method** (**this.checkAnswer(index)**; passing the **index** as an argument). For reasons explained in the last lesson, we must use the JavaScript keyword **this** when referencing one **app** property or method from within the **same** Object (in this case **app**) so that we have the proper **context** (in this case a reference to our parent **app** Object where both the **start** and **checkAnswer** methods live). The other important thing to notice is that we need to attach the JavaScript **bind()** method to each of our Function Definitions. As we explained above, this properly sets the **context** keyword **this** otherwise the call to the **checkAnswer** method will result in an error.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {

  // modified code
  start: function() {
    // existing code
    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    // modified code
    // passing context with bind()
    alts.forEach(function(element, index){

      element.addEventListener('click', function(){
        // new code
        // call checkAnswer method
        this.checkAnswer(index);
      }.bind(this));
    }.bind(this));

    // existing code
    // show first question
    this.showQuestion(question);
  },

  // existing code
  showQuestion: function(q) {
    // show question title
    // show alternatives
  },

  // existing code
  checkAnswer: function(userSelected) {
    // check answer
  }
};
```



Refactor START Method with Arrow Function

Instead of using the **bind()** method attached to a regular **Function Definition** to pass **context**, we can use the JavaScript **Arrow Function**. These compact functions are especially powerful when passing functions as arguments as we do in our **start** method for **adding event listeners** and again for **handling the user clicks** inside that event handler because the **this** context is handled for you by JavaScript. To convert from the Function Definition to the Arrow Function syntax **remove** the word **function** and place an **arrow (=>)** between the arguments and opening body bracket. If there are no arguments, use empty **()** as is the case adding the **click** event handler in the code below. The call to our new method, **checkAnswer (this.checkAnswer(index))**; passing the **index** as an argument), remains the same.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {

  // modified code
  start: function() {
    // existing code
    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    // modified code
    // passing context with arrow function
    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // existing code
    // show first question
    this.showQuestion(question);
  },

  // existing code
  showQuestion: function(q) {
    // show question title
    // show alternatives
  },

  // existing code
  checkAnswer: function(userSelected) {
    // check answer
  }
};
```



Refactor SHOWQUESTION Method

We need to keep track of the current question so we can use it for comparison to the user's selection in the new **checkAnswer** method. A good place to track the current question is in the method we use to show it, **showQuestion**. By using the JavaScript **context** keyword **this** with the variable **currQuestion** we make the variable a property of the **app** Object which makes it available within the **checkAnswer** method.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {
  // existing code
  start: function() {
    // add event handler to check answer
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {
      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // show first question
    this.showQuestion(question);
  },

  // modified code
  showQuestion: function(q) {
    // new code
    // keep track of current question
    this.currQuestion = q;

    // existing code
    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
      element.textContent = q.alternatives[index];
    });
  },

  // new code
  checkAnswer: function(userSelected) {
  }
};
```



```
// initialize the application
app.start();
```

Code CHECKANSWER Method

We now have all the pieces in place to code our new **checkAnswer** method. The **checkAnswer** method receives one **parameter** which is the **index** of the selection the user made from the list of **alternatives**, named **userSelected**. We code an **if** statement to compare this value against the current question **correctAnswer** property (**this.currQuestion.correctAnswer**) from the **question** Object defined at the top of code file and passed to the **showQuestion** method. If the values are equal we **log** the text **correct** to the **console** otherwise we log the text **wrong**.

```
// existing code
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {
  // existing code
  start: function() {
    // add event handler to check answer
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {
      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // show first question
    this.showQuestion(question);
  },

  // existing code
  showQuestion: function(q) {
    // keep track of current question
    this.currQuestion = q;

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
      element.textContent = q.alternatives[index];
    });
  },
}
```

```
// new code
checkAnswer: function(userSelected) {
  if(this.currQuestion.correctAnswer == userSelected) {
    // correct
    console.log('correct');
  }
  else {
    // not correct
    console.log('wrong');
  }
}
};

// initialize the application
app.start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

New Code Flow	Description
let question = { properties:values }	No Change. Define question variable and assign it an Object containing properties and values for title (String), alternatives (Array) and correctAnswer (Number)
app.start method	The start method now only contains logic and functionality required to run one time; when the app starts. This method handles add event handlers, calling the checkAnswer method when a click event happens, and calling the showQuestion method to initiate our user experience by showing the first question
app.showQuestion method	The showQuestion method now keeps track of the current question then shows the question and its alternative answers
app.checkAnswer method	The checkAnswer method is now responsible to responding to the user's answer selection to determine if it is correct

gato

- dog 1
- cat 2
- bird 3
- fish 4

Console

```
>  
wrong 1  
correct 2  
wrong 3  
wrong 4  
> █
```



Up to now we have been testing our code logic using one **question** Object that we defined at the top of our code file. In this lesson, we replace that one **question** Object with an **array** of **question** Objects. We then continue to develop our **app** Object by adding a new method, **increasePosition**, which will help us navigate the **array** of **question** Objects. We will then refactor the **start** and **checkAnswer** methods to use this new **array** and **position** approach.

Add Array of Question Objects

To replace our one **question** Object with an **array** of **question** Objects, you can copy the question list from the course download files for this lesson, copy them from the code block below or code them yourself. Place the array of question Objects at the top of our code file (**script.js**). Feel free to change the questions or add new ones. When we finish this lesson our code is ready to handle any number of questions as long as they are in the same **Object** and **Property:Value** format.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];
```

Refactor START Method

Now that we have an **array** of **question** Objects, we need to refactor our **start** method to use the **questions** array and current position within that array instead of just the one **question** Object we had previously. To do this, we add a new property (**currPosition**) to the **app** Object by using the **this** keyword and **set** its initial **value** to be **zero** (the first question in the array). We then **modify** the call to the **showQuestion** method to pass the **questions** array with **current position** instead of just the original sample **question** Object.

```
// existing code
let questions = [
];
```

```
// modified code
let app = {
```



```
// modified code
start: function () {

  // new code
  this.currPosition = 0;

  // existing code
  // get alternatives
  let alts = document.querySelectorAll('.alternative');

  alts.forEach((element, index) => {

    element.addEventListener('click', () => {
      // check correct answer
      this.checkAnswer(index);
    });
  });

  // modified code
  // show current question
  this.showQuestion(questions[this.currPosition]);
},

// existing code
showQuestion: function (q) {
},

// existing code
checkAnswer: function (userSelected) {
}
};

app.start();
```

Refactor CHECKANSWER Method

Now that we have an **array** of **question** Objects, we need to refactor our **checkAnswer** method to use the **questions** array and current position within that array instead of just the one **question** Object we had previously. To do this, we add a local method variable to hold the current question (**currQuestion**). We set this variable equal to the **current question** which we find by using the **questions** array and our new **app** Object property, **currPosition**. We keep our existing logic to check if we have a correct answer. After this logic block we add two new method calls. The first, a new method called **increasePosition**, which moves to the next question in the array. The second method call is to the existing method, **showQuestion**, to display the new question with its alternatives on the web page.

```
// existing code
let questions = [
];

// modified code
let app = {
  // existing code
```



```
start: function () {
},

// existing code
showQuestion: function (q) {
},

// modified code
checkAnswer: function (userSelected) {

  // new code
  let currQuestion = questions[this.currPosition];

  // existing code
  if (currQuestion.correctAnswer == userSelected) {
    // correct
    console.log('correct');
  }
  else {
    // not correct
    console.log('wrong');
  }

  // new code
  // increase position
  this.increasePosition();

  // show next question
  this.showQuestion(questions[this.currPosition]);
}
};

app.start();
```

Code INCREASEPOSITION Method

Now that we have an **array** of **question** Objects, we need a way to navigate through the **questions** array. Our goal is to **start** with the **first question** in the array (**index of zero**) and be able to **increase** the current **questions** array **index** position by **one** each time this method is called until we reach the end of the array. At that point we want to **reset** the **current position** to the **first question** in the **array** (**index of zero**). The **length** of an **array** can be found in the **array property length**. So to find the length of the questions array we code: **questions.length**. Add the new method, **increasePosition**, inside the **app** Object just as we have with the other methods remembering to separate the methods with a **comma**.

```
// existing code
let questions = [
];

// modified code
let app = {
  // existing code
```



```
start: function () {
},

// existing code
showQuestion: function (q) {
},

// existing code
checkAnswer: function (userSelected) {
},

// new code
increasePosition: function () {
  // increases currPosition by 1
  this.currPosition++;

  // resets currPosition to 0 when end of array is reached
  if (this.currPosition == questions.length) {
    this.currPosition = 0;
  }
}
};

app.start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];
```



```
let app = {

  start: function () {

    this.currPosition = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // show current question
    this.showQuestion(questions[this.currPosition]);
  },

  showQuestion: function (q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function (element, index) {
      element.textContent = q.alternatives[index];
    });
  },

  checkAnswer: function (userSelected) {

    let currQuestion = questions[this.currPosition];

    if (currQuestion.correctAnswer == userSelected) {
      // correct
      console.log('correct');
    }
    else {
      // not correct
      console.log('wrong');
    }

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
  },
```



```
increasePosition: function () {  
  // increases currPosition by 1  
  this.currPosition++;  
  
  // resets currPosition to 0 when end of array is reached  
  if (this.currPosition == questions.length) {  
    this.currPosition = 0;  
  }  
}  
};  
  
app.start();
```

New Code Flow

let questions = [{ properties:values }];

app.start method

app.showQuestion method
app.checkAnswer method

app.increasePosition method

Description

Changed to array of questions. Define **questions** variable and assign it an **array** of **question** Objects each containing properties and values for **title** (String), **alternatives** (Array) and **correctAnswer** (Number)

Modified. The **start** method now contains logic and functionality required to use the **questions** array and **current position** within that array
No Change.

Modified. The **checkAnswer** method now contains logic and functionality required to use the **questions** array and **current position** within that array. New method calls were added to move to the next question then show that next question on the web page

Added. Now that we have an **array** of **question** Objects, this new method provides a way to navigate through the **questions** array by adjusting the array's current **index** position

The web page **updates** to a **new question** and **alternatives** each time we **select** an **answer**. If we simply test click on the next **alternative** each time the **question changes** we see the following **results** in the **console**. For example, select the first alternative for question one, the second alternative for question two and so on.

gato

- dog 1
- cat
- bird
- fish

ave

- mouse
- hamster 2
- lizard
- bird

rata

- cat
- fish
- rat 3
- shark

mosca

- fly
- puma
- fish
- dog 4

Console

```
>  
wrong 1  
wrong 2  
correct 3  
wrong 4  
> █
```



In this lesson, we add the functionality to keep **track** of the user's **score** and **show** that **score** on the **web page**. We do this by adding a new **HTML** element to our **index.html** web page to show the **score**. The functionality to **show** this **score** will be coded in a new method, **updateStats**, inside our **app** Object. We will then refactor the **start** method to add a **score** variable to the **app** Object as well as call our new **updateStats** method so the score is available when the first question is shown. Next, we refactor the **checkAnswer** method to increase the score if the answer is correct and then refresh the score on the web page by calling our new **updateStats** method.

Add Score HTML Element to Web Page

The first step in showing the **score** is to have a place to show it. For this we will add a **<div>** with an **id** of **score** to our **index.html** file just below the list of alternatives so we can reference it from **JavaScript** to update **content** and from **CSS** to add **styling**.

```
<!-- existing code -->
<div id="title"></div>
<ul>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
</ul>
<!-- new code -->
<div id="score"></div>

<script src="script.js"></script>
```

Code UPDATESTATS Method

Now that we have a new **HTML** element to show the **score** we add the functionality to **show** this **score** in a new method, **updateStats**, inside our **app** Object. The goal of the code is to show the score to the user on the web page along with the text **Your Score**. To accomplish this we follow the familiar pattern of **selecting** and then **modifying** the HTML element, **<div>** with an **id** of **score**, which we added to our **index.html** file. Add the new method, **updateStats**, inside the **app** Object just as we have with the other methods remembering to separate the methods with a **comma**.

```
// existing code
let questions = [
];

// modified code
let app = {
  // existing code
  start: function () {
  },

  // existing code
  showQuestion: function (q) {
  },

  // existing code
  checkAnswer: function (userSelected) {
  },
```



```
// existing code
increasePosition: function () {
},

// new code
updateStats: function () {
  // select score div element
  let scoreDiv = document.getElementById('score');
  // modify score div element
  scoreDiv.textContent = `Your score: ${this.score}`;
}
};

app.start();
```

Notice the use of JavaScript **Template Literals** syntax which allows us to combine **static** and **dynamic content** in the same **text** string. Template Literals are enclosed by the backtick character. Inside the backtick characters, **dynamic** content is enclosed by a dollar sign (**\$**) and curly braces (**{ }**).

Refactor START Method

Now that we can display the score to the user on the web page, we need to add logic to **track** the **score** as well as functionality to **call** our new **updateStats** method to display the latest score. First, in the **start** method, we add a new property (**score**) to the **app** Object by using the **this** keyword and **set** its initial **value** to be **zero**. We then call to the **updateStats** method before calling the method to show the question, **showQuestion**.

```
// existing code
let questions = [
];

// modified code
let app = {
  // modified code
  start: function () {

    // existing code
    this.currPosition = 0;
    // new code
    this.score = 0;

    // existing code
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {
      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });
  });
};
```



```
// new code
this.updateStats();

// existing code
this.showQuestion(questions[this.currPosition]);
},

// existing code
showQuestion: function (q) {
},

// existing code
checkAnswer: function (userSelected) {
},

// existing code
increasePosition: function () {
},

// existing code
updateStats: function () {
  let scoreDiv = document.getElementById('score');
  scoreDiv.textContent = `Your score: ${this.score}`;
}
};

app.start();
```

Refactor CHECKANSWER Method

Now that we can display the score to the user on the web page and we have a **score** property on our **app** Object, we need to add logic to **increase** the **score** (if the answer is correct) as well as functionality to **call** our new **updateStats** method to display the latest score.

```
// existing code
let questions = [
];

// modified code
let app = {
  // existing code
  start: function () {
  },

  // existing code
  showQuestion: function (q) {
  },

  // modified code
  checkAnswer: function (userSelected) {
    // existing code
    let currQuestion = questions[this.currPosition];
    // modified code
```



```
    if (currQuestion.correctAnswer == userSelected) {
      // existing code
      console.log('correct');
      // new code (add 1 to score)
      this.score++;
    }
    else {
      console.log('wrong');
    }

    // new code
    this.updateStats();

    // existing code
    this.increasePosition();

    // existing code
    this.showQuestion(questions[this.currPosition]);
  },

  // existing code
  increasePosition: function () {
  },

  // existing code
  updateStats: function () {
  }
};

app.start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
```



```
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];

let app = {
  start: function () {

    this.currPosition = 0;
    this.score = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // refresh stats
    this.updateStats();

    // show first question
    this.showQuestion(questions[this.currPosition]);
  },

  showQuestion: function (q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function (element, index) {
      element.textContent = q.alternatives[index];
    });
  },

  checkAnswer: function (userSelected) {

    let currQuestion = questions[this.currPosition];

    if (currQuestion.correctAnswer == userSelected) {
      // correct
      console.log('correct');
      // increase score by 1
      this.score++;
    }
  }
}
```



```
    else {
      // not correct
      console.log('wrong');
    }

    // refresh stats
    this.updateStats();

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
  },

  increasePosition: function () {
    // increase index to questions array
    this.currPosition++;

    if (this.currPosition == questions.length) {
      this.currPosition = 0;
    }
  },

  updateStats: function () {
    // show latest score on web page
    let scoreDiv = document.getElementById('score');
    scoreDiv.textContent = `Your score: ${this.score}`;
  },

};

app.start();
```

New Code Flow

let questions = [{ properties:values }];
app.start method

app.showQuestion method
app.checkAnswer method

app.increasePosition method
app.updateStats method

Description

No Change.

Modified. The **start** method now contains logic and functionality required to **track** the **score** and call the method, **updateStats**, to refresh it on the web page

No Change.

Modified. The **checkAnswer** method now contains logic and functionality required to **increase** the **score** (if answer is correct) and call the method, **updateStats**, to refresh it on the web page

No Change.

Added. Display the latest **score** on the web page

The web page **updates** to a **new question** and **alternatives** each time we **select** an **answer**. Now the updated **score** shows below the list of alternatives, increasing by one if the answer is correct.

gato

- dog
- cat
- bird
- fish

1

Your score: 0

Console

```
> correct 1
> 
```

ave

- mouse
- hamster
- lizard
- bird

Your score: 1



In this lesson, we add the functionality to **show** the user a **message** indicating if they got the answer correct and if not, showing them the correct answer. We do this by adding a new **HTML** element to our **index.html** web page to show the **result**. The functionality to **show** the **result** will be coded in a new method, **showResult**, inside our **app** Object. We will then refactor the **checkAnswer** method to call our new **showResult** method passing a boolean value indicating if they got the answer correct.

Add Result HTML Element to Web Page

The first step in showing the **result** is to have a place to show it. For this we will add a **<div>** with an **id** of **result** to our **index.html** file just below the list of alternatives so we can reference it from **JavaScript** to update **content** and from **CSS** to add **styling**.

```
<!-- existing code -->
<div id="title"></div>
<ul>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
</ul>
<!-- new code -->
<div id="result"></div>
<!-- existing code -->
<div id="score"></div>

<script src="script.js"></script>
```

Code SHOWRESULT Method

Now that we have a new **HTML** element to show the **result** we add the functionality to **show** the **result** in a new method, **showResult**, inside our **app** Object. The goal of the code is to **show** the user a **message** indicating if they got the answer correct and if not, showing them the correct answer. To accomplish this we follow the familiar pattern of **selecting** and then **modifying** the HTML element, **<div>** with an **id** of **result**, which we added to our **index.html** file. Add the new method, **showResult**, inside the **app** Object just as we have with the other methods remembering to separate the methods with a **comma**. This method receives one parameter, **isCorrect**, which we use to determine which message to show the user.

```
// existing code
let questions = [
];

// modified code
let app = {
  // existing code
  start: function () {
  },

  // existing code
  showQuestion: function (q) {
  },
```



```
// existing code
checkAnswer: function (userSelected) {
},

// existing code
increasePosition: function () {
},

// existing code
updateStats: function () {
},

// new code
showResult: function (isCorrect) {
  let resultDiv = document.getElementById('result');
  let result = '';

  // determine which result message to show
  if (isCorrect) {
    result = 'Correct Answer!';
  }
  else {
    // get the current question
    let currQuestion = questions[this.currPosition];

    // get correct answer (index)
    let correctAnswIndex = currQuestion.correctAnswer;

    // get correct answer (text)
    let correctAnswText = currQuestion.alternatives[correctAnswIndex];

    result = `Wrong! Correct answer: ${correctAnswText}`;
  }

  resultDiv.textContent = result;
}
};

app.start();
```

As you can see, this **code** is **not as simple** as just **selecting** the HTML element and **modifying** its value. We start by **selecting** the `<div>` with an **id** of **result** in the normal way **storing** its **value** in a local variable we call **resultDiv**. We then define the **result** variable (String) we will use to modify the **resultDiv** content setting it initially to an empty string. The next step is to determine if the user answered the question correctly using the passed in parameter, **isCorrect**. If **true**, then we simply assign the text **Correct Answer!** to the **result** variable and update the **resultDiv** **textContent** property which updates the web page. If the parameter, **isCorrect**, is **false** we have additional work to do to form the result message. These steps are defined in the table below:

Code Step

```
let currQuestion = questions[this.currPosition];
```

Description

Get the current question from the **questions** array using the current position property (**currPosition**) stored on the **app** Object



```
let correctAnswIndex =  
currQuestion.correctAnswer;
```

```
let correctAnswText =  
currQuestion.alternatives[correctAnswIndex];
```

```
result = `Wrong! Correct answer:  
${correctAnswText}`;
```

Using the current question (**currQuestion**) and its correct answer property (**correctAnswer**), we get the **correct answer index** which tells us the **position** of the correct answer in the **alternatives** array property

Using the correct answer index (**correctAnswIndex**) we get the **correct answer text** from the current question's **alternatives** array property

We can now use Template Literal syntax to assign the **result** variable the **static** text **Wrong! Correct answer:** joined with the **dynamic** correct answer text (**correctAnswText**). This **result** variable is then used to update the **resultDiv.textContent** property which updates the web page

Refactor CHECKANSWER Method

Refactor the **checkAnswer** method to **add** the functionality required to **call** the **showResult** method passing a **boolean value** to indicate if the user answered the question correctly.

```
// existing code  
let questions = [  
];  
  
// modified code  
let app = {  
  // existing code  
  start: function () {  
  },  
  
  // existing code  
  showQuestion: function (q) {  
  },  
  
  // modified code  
  checkAnswer: function (userSelected) {  
    // existing code  
    let currQuestion = questions[this.currPosition];  
  
    // modified code  
    if (currQuestion.correctAnswer == userSelected) {  
      // existing code  
      console.log('correct');  
      this.score++;  
      // new code  
      this.showResult(true);  
    }  
    else {  
      // existing code  
      console.log('wrong');  
      // new code  
      this.showResult(false);  
    }  
  }  
};
```



```
// existing code
this.updateStats();

// existing code
this.increasePosition();

// existing code
this.showQuestion(questions[this.currPosition]);
},

// existing code
increasePosition: function () {
},

// existing code
updateStats: function () {
},

// existing code
showResult: function (isCorrect) {
}
};

app.start();
```

Our Updated Architecture

With this iteration of refactoring complete, let's consider our new code flow, web page functionality, and console output.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];
```



```
let app = {
  start: function () {

    this.currPosition = 0;
    this.score = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // refresh stats
    this.updateStats();

    // show first question
    this.showQuestion(questions[this.currPosition]);
  },

  showQuestion: function (q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function (element, index) {
      element.textContent = q.alternatives[index];
    });
  },

  checkAnswer: function (userSelected) {

    let currQuestion = questions[this.currPosition];

    if (currQuestion.correctAnswer == userSelected) {
      // correct
      console.log('correct');
      this.score++;
      this.showResult(true);
    }
    else {
      // not correct
      console.log('wrong');
      this.showResult(false);
    }
  }
}
```



```
// refresh stats
this.updateStats();

// increase position
this.increasePosition();

// show next question
this.showQuestion(questions[this.currPosition]);
},

increasePosition: function () {
  this.currPosition++;

  if (this.currPosition == questions.length) {
    this.currPosition = 0;
  }
},

updateStats: function () {
  // show score on web page
  let scoreDiv = document.getElementById('score');
  scoreDiv.textContent = `Your score: ${this.score}`;
},

showResult: function (isCorrect) {
  let resultDiv = document.getElementById('result');
  let result = '';

  // checks
  if (isCorrect) {
    result = 'Correct Answer!';
  }
  else {
    // get the current question
    let currQuestion = questions[this.currPosition];

    // get correct answer (index)
    let correctAnswIndex = currQuestion.correctAnswer;

    // get correct answer (text)
    let correctAnswText = currQuestion.alternatives[correctAnswIndex];

    result = `Wrong! Correct answer: ${correctAnswText}`;
  }

  // show result text on web page
  resultDiv.textContent = result;
}
};

app.start();
```

New Code Flow

```
let questions = [ { properties:values } ];
```

Description

No Change.



app.start method

app.showQuestion method

app.checkAnswer method

No Change.

No Change.

Modified. The **checkAnswer** method now contains functionality required to **call** the **showResult** method passing a boolean value to indicate if the user answered the question correctly

app.increasePosition method

app.updateStats method

app.showResult method

No Change.

No Change.

Added. Display the **result message** on the web page indicating to the user if they answered the question correctly. If they did not, the correct answer is included in the response

The web page **updates** to a **new question** and **alternatives** each time we **select** an **answer**. Now the **result message** and updated **score** shows below the list of **alternatives**.

gato

- dog
- cat
- bird
- fish

1 (correct answer)

Your score: 0

ave

- mouse
- hamster
- lizard
- bird

2 (incorrect answer)

Correct Answer!

Your score: 1

rata

- cat
- fish
- rat
- shark

Wrong! Correct answer: bird

Your score: 1

Console

```
>
correct 1
wrong 2
> 
```

In this lesson we will finalize our app. This requires three steps. First, we will convert our web page to a proper website HTML document. Second, we will make a small but important change to the structure of our web page which will enable us to better style our app. Finally, we will add the CSS styling to create the look and feel we want for our app.

Web Page to Website HTML Document

To make our **web page** a **standard HTML website document** we add several HTML elements that provide information to the web browser. These elements are described in the comments below and the code can be copied from the course downloads for this lesson, copied from the code block below or keyed into your editor. Notice that within the **HTML <body> element** is all of the **code** previously present in our **index.html** web page document. Modify the **index.html** file to include the following code:

```
<!--Define the document type-->
<!DOCTYPE html>

<!--Define the container for all HTML elements-->
<html>

  <!--Define the container for metadata-->
  <head>
    <!--Define metadata-->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width">
    <meta name="description" content="JS Language Learning Game - Styling the App">
    <!-- title for the browser tab -->
    <title>Learn JS at Zenva</title>
    <!-- stylesheet link -->
    <link rel="stylesheet" href="style.css">
  </head>

  <body>
    <div id="title"></div>
    <ul>
      <li class="alternative"></li>
      <li class="alternative"></li>
      <li class="alternative"></li>
      <li class="alternative"></li>
    </ul>
    <div id="result"></div>
    <div id="score"></div>
    <script src="script.js"></script>
  </body>
</html>
```

Refactor HTML Structure for Styling

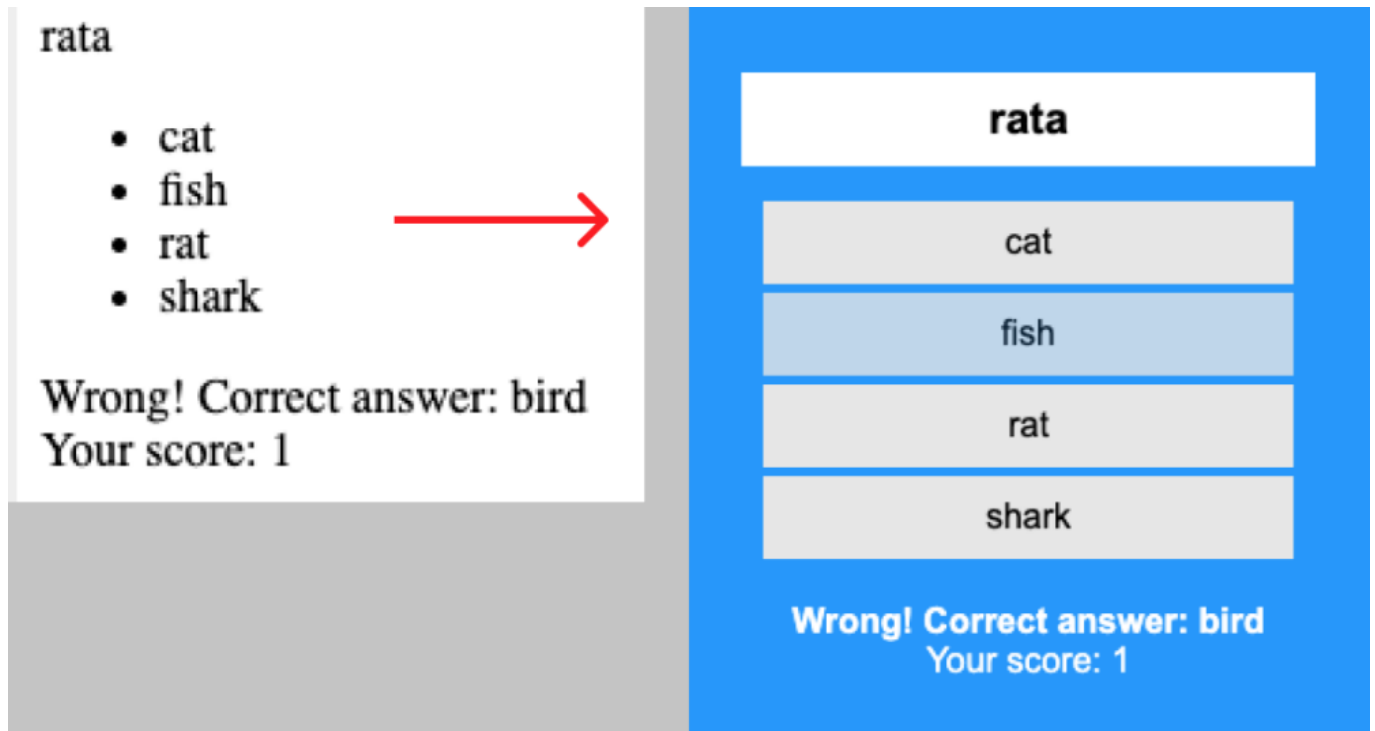
In the **HTML** code above, we want to make one small structural change inside the **HTML <body> element** which will enable us to better style the app. Inside the **index.html** file, wrap our **<body>** elements in a **<div>** with an **id** of **wrapper**. We can then target this element from our **CSS stylesheet (style.css)**. The importance of this common practice will be more obvious in the next section.



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <!-- existing code -->
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <div id="wrapper">
      <div id="title"></div>
      <ul>
        <li class="alternative"></li>
        <li class="alternative"></li>
        <li class="alternative"></li>
        <li class="alternative"></li>
      </ul>
      <div id="result"></div>
      <div id="score"></div>
    </div>
    <script src="script.js"></script>
  </body>
</html>
```

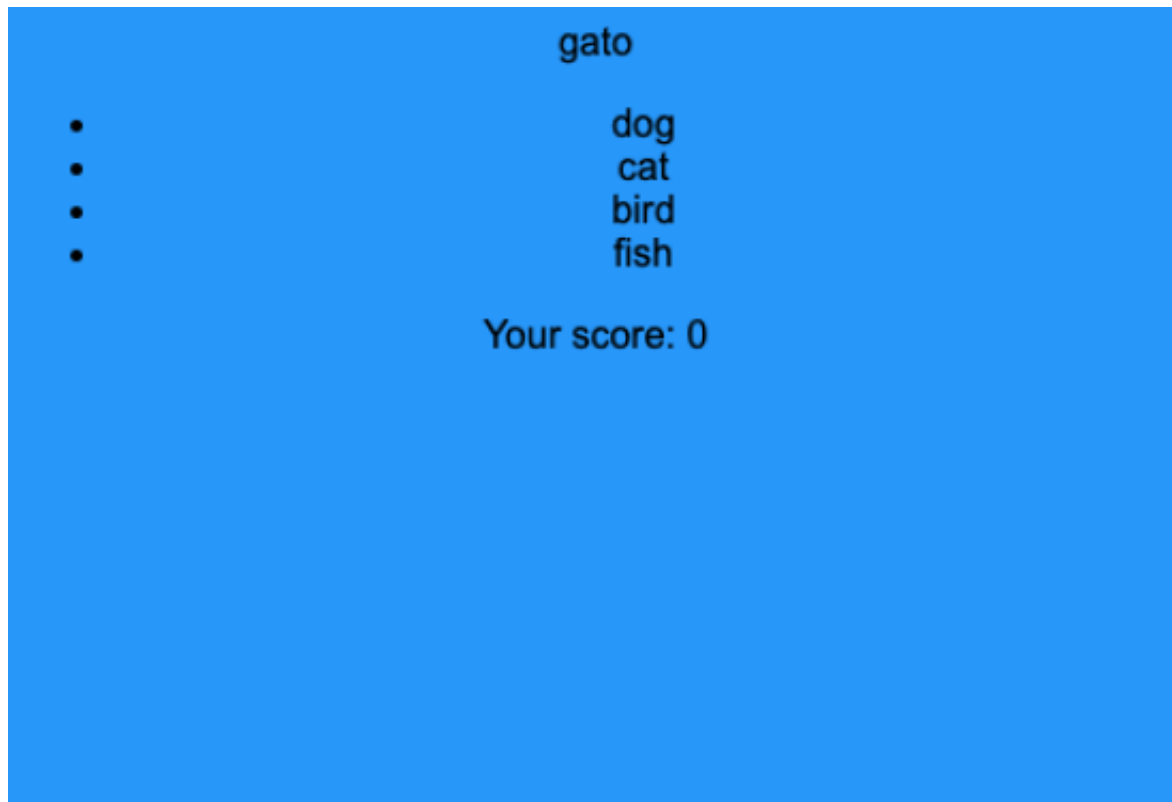
CSS Styles

In the **HTML** code above, we **link** to a **CSS stylesheet** (**style.css**). Create this file in the same **folder** as the **web page** file (**index.html**). In **style.css** we will place all the **CSS styling** to transform the look and feel of our app. In previous lesson we have used the **browser's developer tools** to view the **console**. In this lesson we will use another **tab** named **Elements** which allow you to view the HTML elements on your web page. With the app running in the browser and developer tools open to **Elements** let's begin to transform the look and feel of our app. The image below shows the before and after.



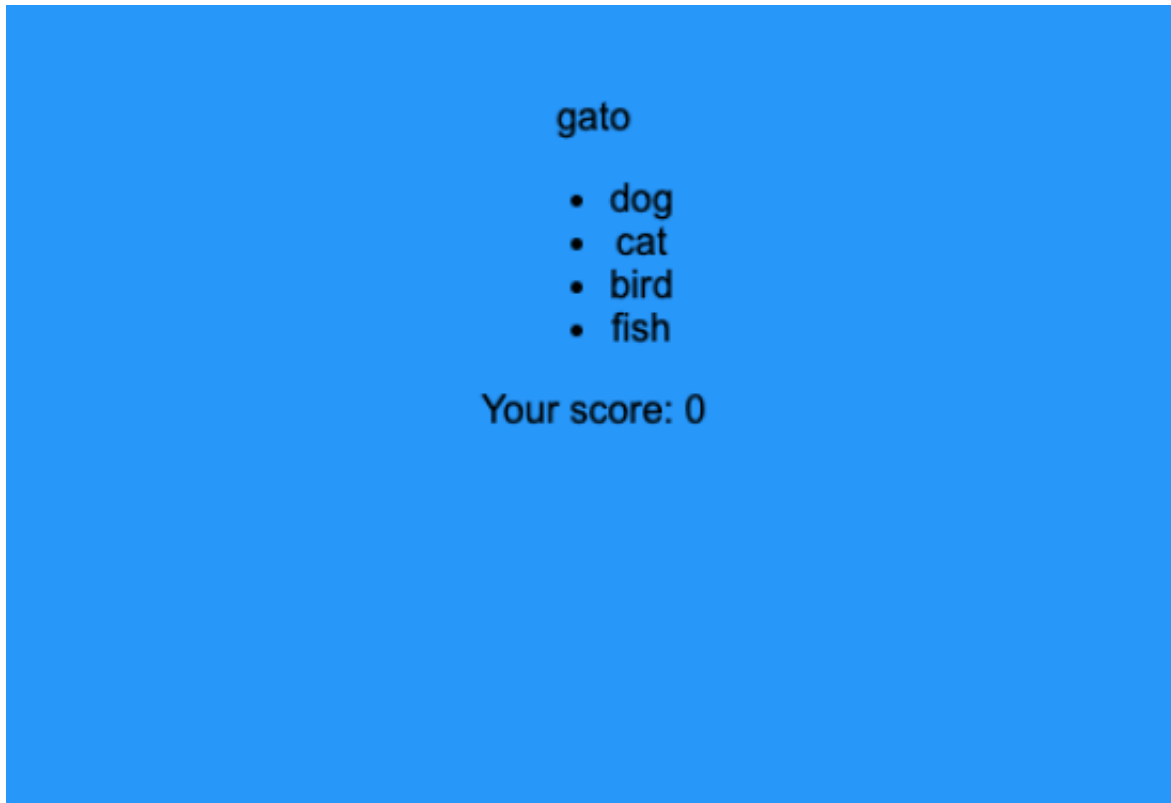
We begin inside **style.css** with some basic styles to be applied to the **<body>** of our web page. This CSS code targets the **<body>** tag and applies a background color, font and aligns our text content to the horizontal center of the web page.

```
body {  
  background: #1E90FF;  
  font-family: arial;  
  text-align: center;  
}
```



We next target the `<div>` with an **id** of **wrapper** which is the main **container** for our app's display elements. To target an element by **id** in **CSS** we use the number sign (`#`) followed by the **id name** (`#wrapper`). In this code we provide some **margin** on **top** to move our content away from the top of the web page and set the **width** of our app to take up the **entire web page display** width (viewport). The other important code here tells our **container** to act as a **CSS Flexbox** which allows us to effectively **arrange** our **content**. In this case we arrange the sections within the **wrapper** (question, alternatives, results message, score) in a **column** (stacked vertically) and then **center** that column within the **wrapper**.

```
#wrapper {  
  margin-top: 40px;  
  width: 100%;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```



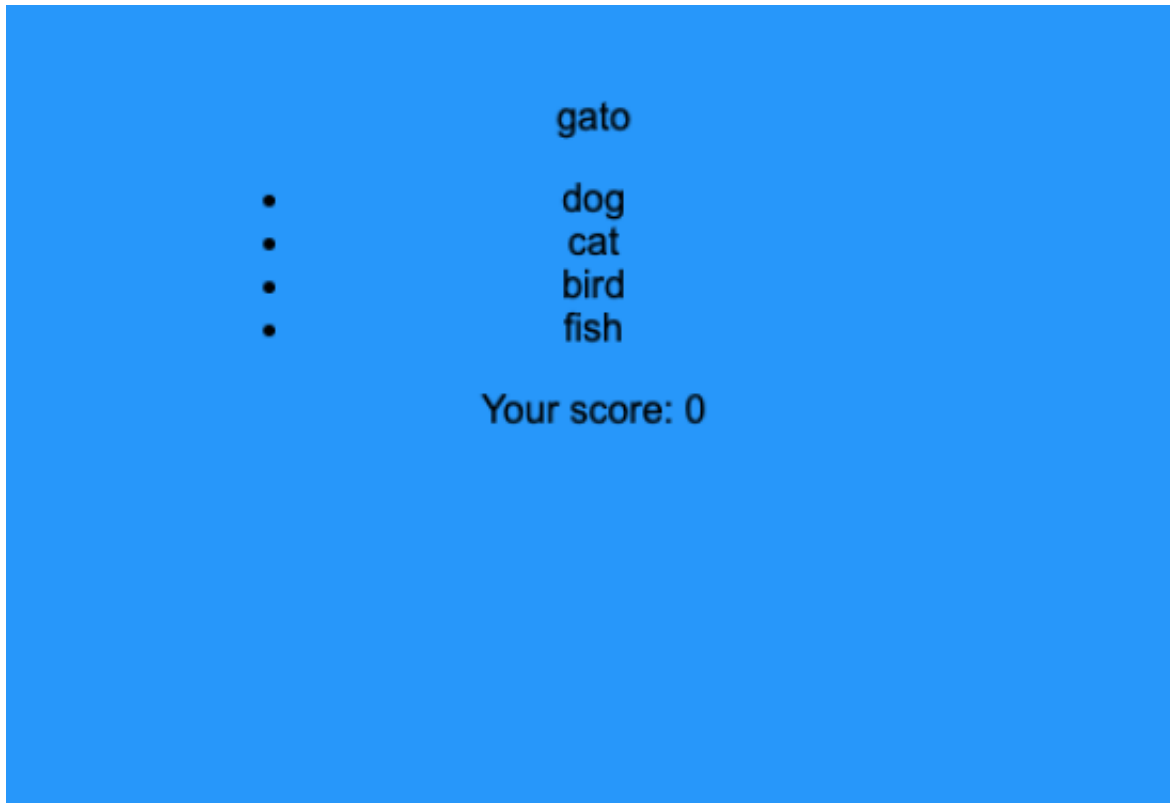
We next target all **<div>** elements as well as the List container **** inside the **<div>** with an **id** of **wrapper**. We can target the nested elements by placing them after the **#wrapper** in the CSS selector, separating each selector with a **comma**. We style the List container to take up half of the wrapper container up to a maximum width of 300 pixels.

```
#wrapper div, #wrapper ul {  
  width: 50%;  
  max-width: 300px;  
}
```



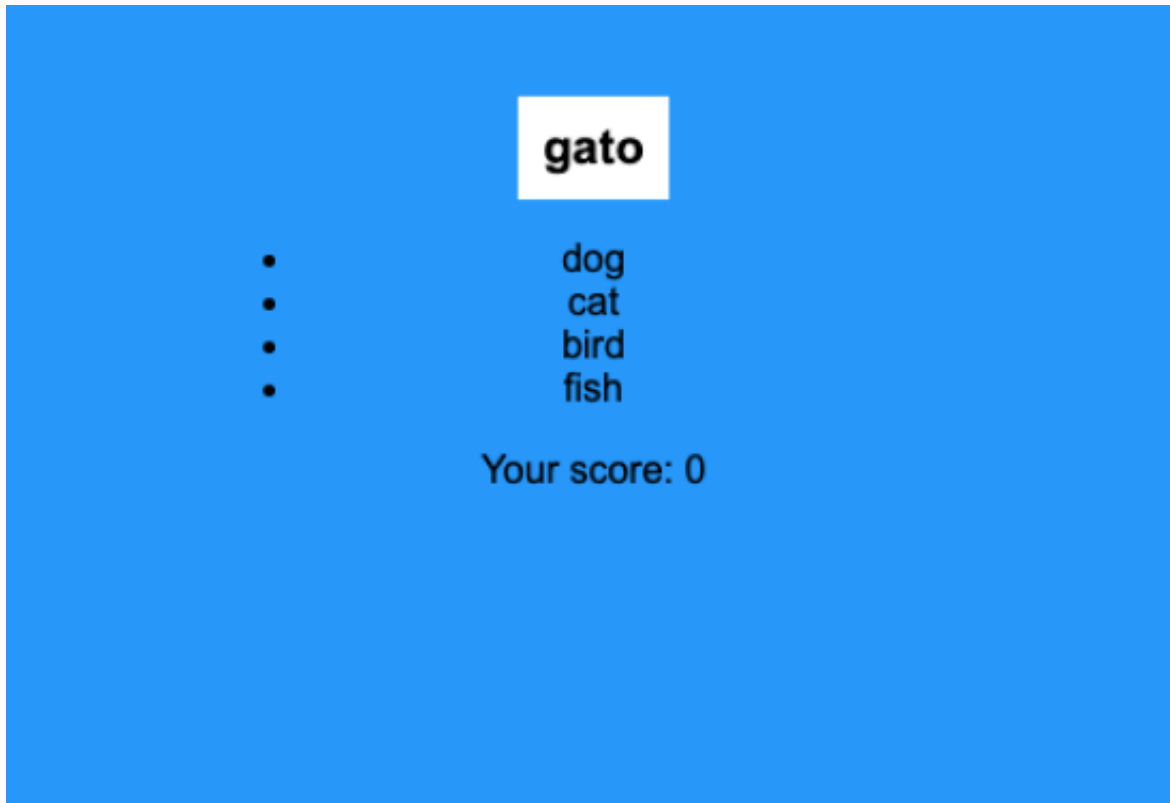
We next target the List container `<ul>` itself to **remove** the **default padding** that the browser applies. We can notice a small shift in the **alternatives** text.

```
ul {  
  padding: 0px;  
}
```



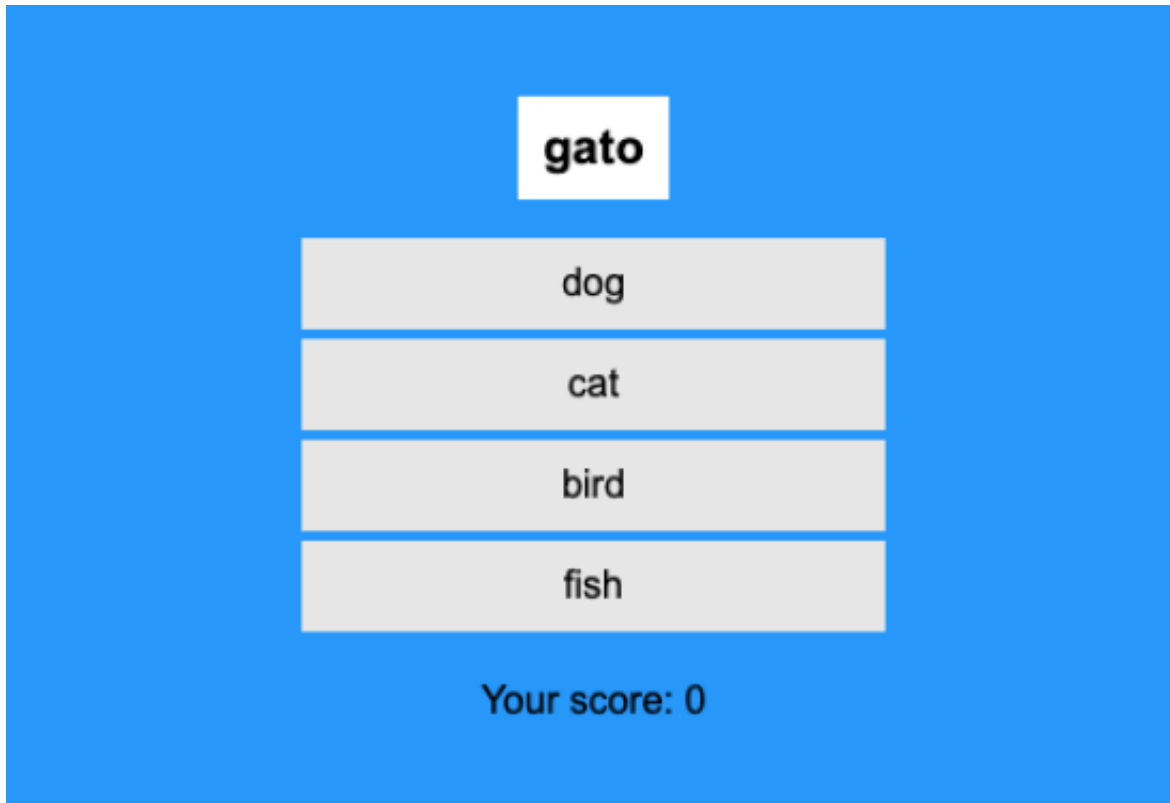
We now begin to target individual sections of our app starting with the **question** which we show inside the `<div>` with an **id** of **title**. We adjust its text size, make it bold and provide a background with some padding so that the question stands out.

```
#title {  
  font-size: 20px;  
  font-weight: bold;  
  background: white;  
  padding: 10px;  
}
```



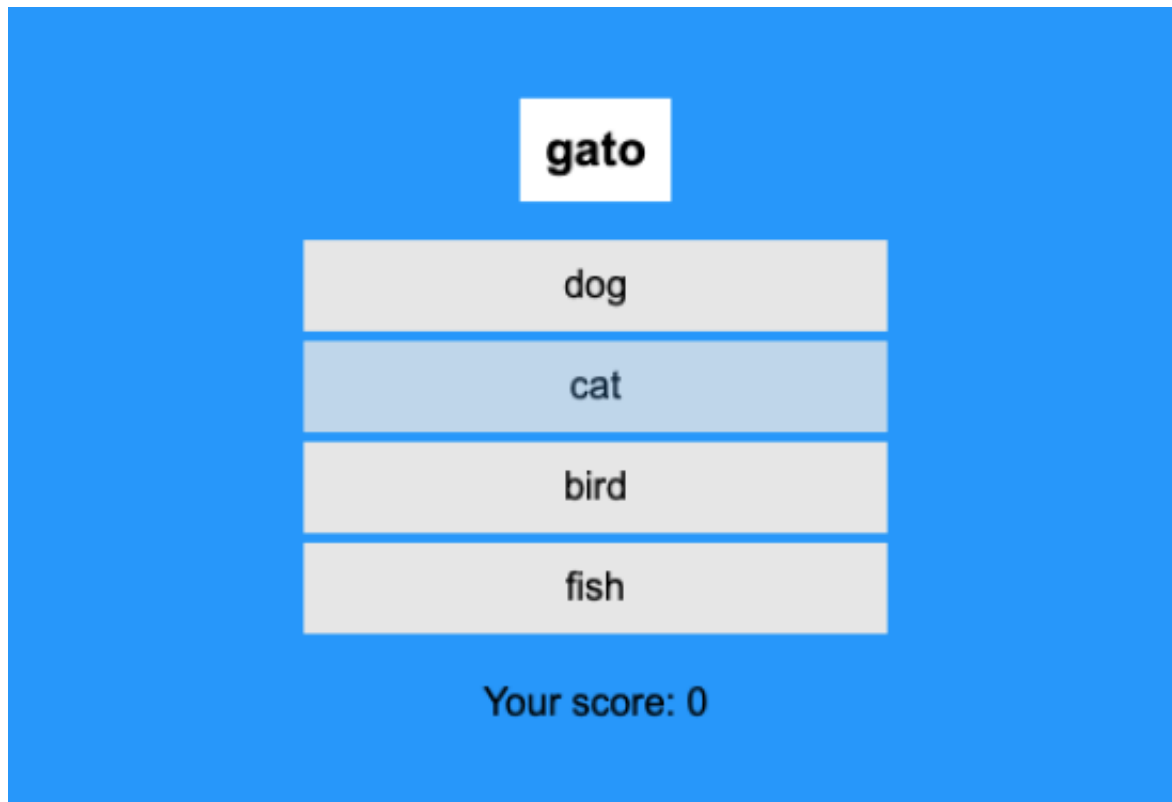
We now target the individual **alternatives** each of which we show inside List Item `<li>` elements with a **CSS class** of **alternative**. We target a **class** in **CSS** by using a **period** followed by the **class name** (**.alternative**). We change the list style to be none which removes the default list bullets, provide a background color and separate each item so they look like individual buttons.

```
.alternative {  
  list-style-type: none;  
  margin-bottom: 4px;  
  padding: 10px;  
  background: #E6E6E6;  
}
```



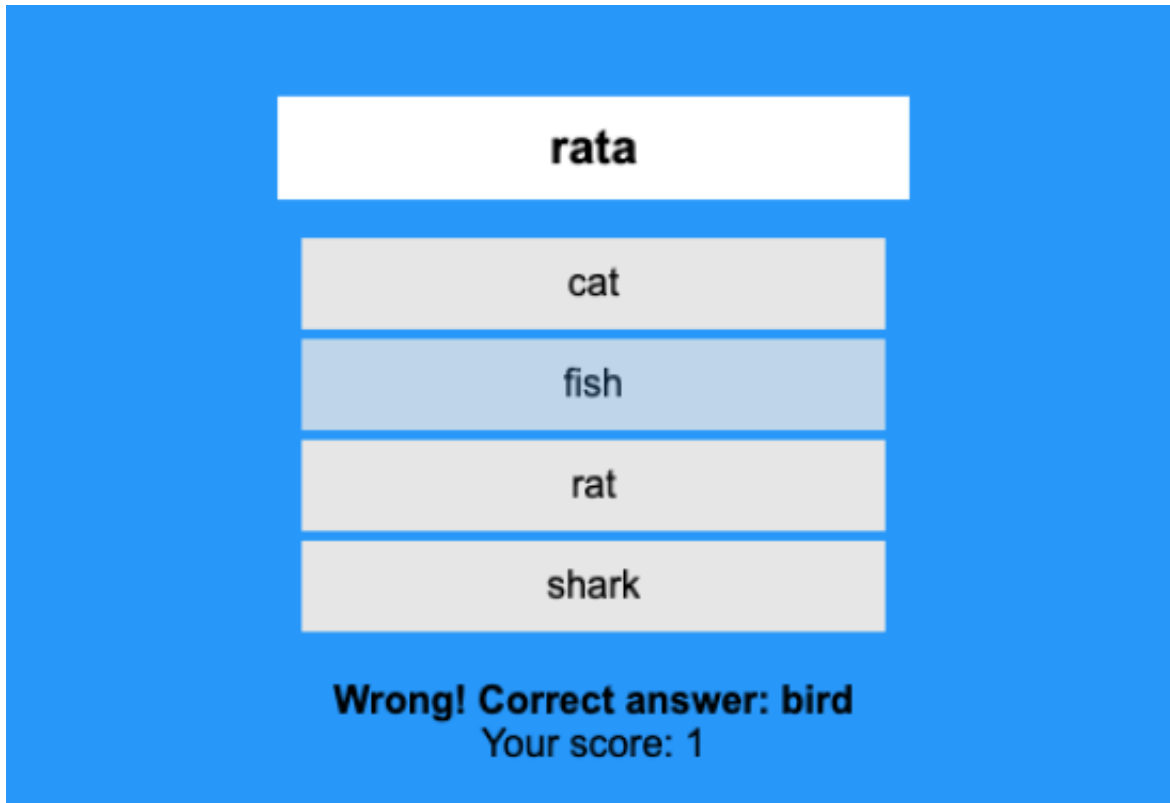
Next, we add an effect to the **alternatives** so that the appearance of a single alternative changes slightly when the mouse hovers over it. We add **:hover** to **.alternative** class selector to trigger this effect. This is an example of what is known as a **CSS pseudo-class**. When the mouse hovers over an alternative the mouse cursor changes to be a pointer and we make the element slightly transparent to create an effect.

```
.alternative:hover {  
  cursor: pointer;  
  opacity: 0.8;  
}
```



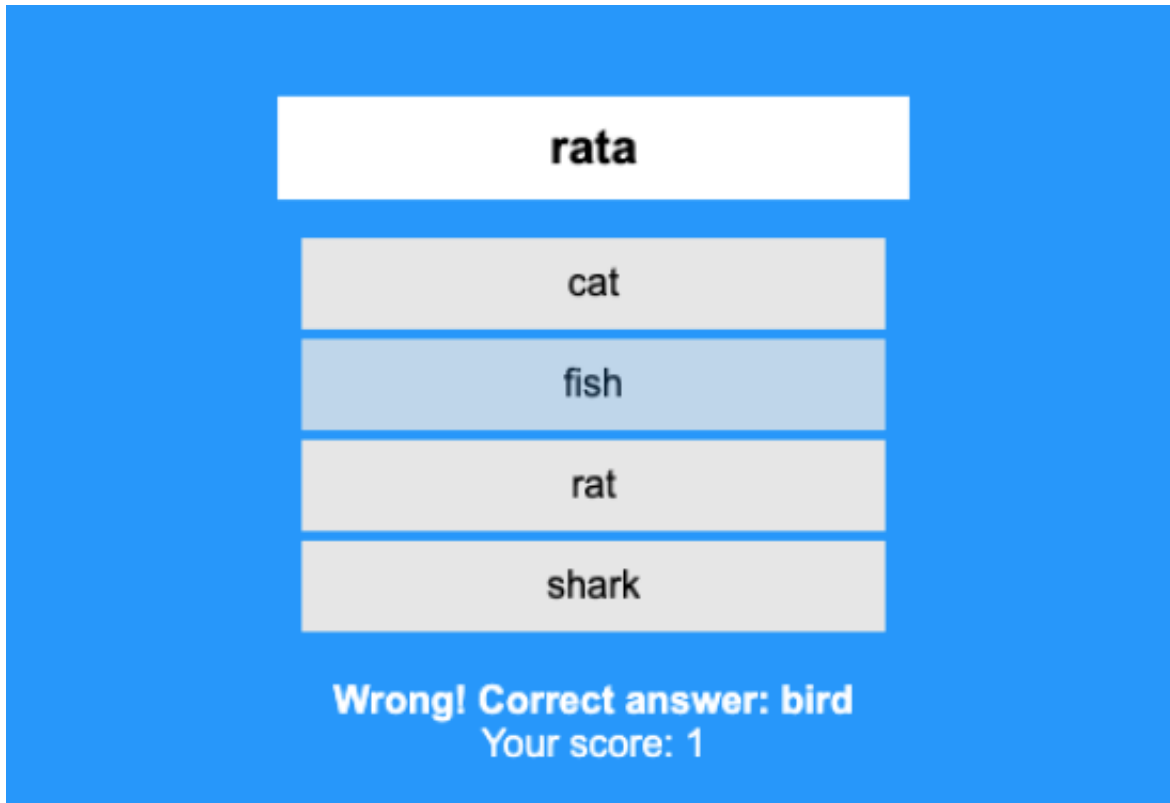
We now target the **result message** which we show inside the `<div>` with an **id** of **result** and make its text bold.

```
#result {  
  font-weight: bold;  
}
```



Finally, we target both the **score** (`#score`) and **result message** (`#result`) to change their text color to white.

```
#score, #result {  
  color: white;  
}
```



As you can see, a little bit of CSS styling and our app looks better. Of course, this is just a starting point. Congratulations on building this language learning app with JavaScript, HTML and CSS from scratch!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Showing a Question

index.html

Found in Project/Project 01 - Showing a Question

This is an HTML code snippet that creates a title placeholder and an unordered list with four list items, each with the class name “alternative”. A JavaScript file named ‘script.js’ is referenced, which manipulates or populates these HTML elements based on its included code.

```
<div id="title"></div>
<ul>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
</ul>

<script src="script.js"></script>
```

script.js

Found in Project/Project 01 - Showing a Question

This JavaScript code defines a question object with title, alternatives, and the correct answer. The function showQuestion displays the question title in a specific HTML element identified with the ID ‘title’. It also assigns each alternative from the question object to elements with the class ‘alternative’.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

function showQuestion(q) {
  // 1. select dom element
  let titleDiv = document.getElementById('title');

  // 2. modify it
  titleDiv.textContent = q.title;

  // selecting by a query
  let alts = document.querySelectorAll('.alternative');

  alts.forEach(function(element, index){
```

```
        element.textContent = q.alternatives[index];
    });
}

showQuestion(question);
```

Handling User Input

index.html

Found in Project/Project 02 - Handling User Input

This code creates an HTML structure consisting of a title, a list with four items, and a button. It also links an external JavaScript file "script.js" which likely contains functionality for the HTML elements.

```
<div id="title"></div>
<ul>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
  <li class="alternative"></li>
</ul>

<button id="btn">Click Me</button>

<script src="script.js"></script>
```

script.js

Found in Project/Project 02 - Handling User Input

This JavaScript code defines a question object and a function to display the question and its alternatives in a webpage. The code also adds click event listeners to each alternative, displaying whether the selected option is correct. Finally, a message is output to the console when a button is clicked.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

function showQuestion(q) {
  // show question title
  let titleDiv = document.getElementById('title');
  titleDiv.textContent = q.title;

  // show alternatives
  let alts = document.querySelectorAll('.alternative');

  alts.forEach(function(element, index){
    element.textContent = q.alternatives[index];
```

```
        element.addEventListener('click', function(){
            // check correct answer
            if(q.correctAnswer == index) {
                console.log('Correct Answer!');
            }
            else {
                console.log('Wrong Answer!');
            }
        });
    });
}

showQuestion(question);

let btn = document.getElementById('btn');

btn.addEventListener('click', function(){
    console.log('Clicked!');
});
```

App Structure - Part 1

index.html

Found in Project/Project 03 - App Structure Part 1

This HTML code sets up a placeholder for a title and a list with four list items. These elements are intended to be used for displaying a question and multiple-choice alternatives, with their behavior defined in an external JavaScript file called “script.js”.

```
<div id="title"></div>
<ul>
    <li class="alternative"></li>
    <li class="alternative"></li>
    <li class="alternative"></li>
    <li class="alternative"></li>
</ul>

<script src="script.js"></script>
```

script.js

Found in Project/Project 03 - App Structure Part 1

This JavaScript code defines a question object along with a ‘start’ function which sets up click event listeners for multiple-choice alternatives. It also includes a ‘showQuestion’ function that displays the question title and alternatives on a webpage. The ‘start’ function then initiates this process.

```
let question = {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
```



```
    correctAnswer: 1
};

function start() {
    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.addEventListener('click', function(){
            // check correct answer
            console.log('check correct answer');
        });
    });

    // show first question
    showQuestion(question);
}

function showQuestion(q) {
    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.textContent = q.alternatives[index];
    });
}

start();
```

App Structure - Part 2

script.js

Found in Project/Project 04 - App Structure Part 2

This JavaScript code forms an ‘app’ object that includes methods to start an application displaying a multiple-choice question, and to show the question title and alternatives. An event listener is also set up to check for the correct answer when an alternative is clicked. The ‘start’ method is then called to initiate the process.

```
let question = {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
};

let app = {
    start: function() {
```



```
// get alternatives
let alts = document.querySelectorAll('.alternative');

alts.forEach(function(element, index){
  element.addEventListener('click', function(){
    // check correct answer
    console.log('check correct answer');
  });
});

// show first question
this.showQuestion(question);
},

showQuestion: function(q) {
  // show question title
  let titleDiv = document.getElementById('title');
  titleDiv.textContent = q.title;

  // show alternatives
  let alts = document.querySelectorAll('.alternative');

  alts.forEach(function(element, index){
    element.textContent = q.alternatives[index];
  });
}
};

app.start();
```

Context and Checking the Answer

script.js

Found in Project/Project 05 - Context and Checking Answer

The JavaScript code constructs an application with the ability to display a multiple-choice question, check user's selection against the correct answer, and provide feedback. The app object has three methods declared - 'start', 'showQuestion', and 'checkAnswer'. It uses arrow functions to preserve the 'this' context inside nested functions.

```
let question = {
  title: 'gato',
  alternatives: ['dog', 'cat', 'bird', 'fish'],
  correctAnswer: 1
};

let app = {
  start: function() {

    // get alternatives
    let alts = document.querySelectorAll('.alternative');
```



```
//      alts.forEach(function(element, index){
//          console.log(this);
//
//          element.addEventListener('click', function(){
//              // check correct answer
//              this.checkAnswer(index);
//          }.bind(this));
//      }.bind(this));

      alts.forEach((element, index) => {

          element.addEventListener('click', () => {
              // check correct answer
              this.checkAnswer(index);
          });
      });

      // show first question
      this.showQuestion(question);
  },

  showQuestion: function(q) {

      // keep track of the current question
      this.currQuestion = q;

      // show question title
      let titleDiv = document.getElementById('title');
      titleDiv.textContent = q.title;

      // show alternatives
      let alts = document.querySelectorAll('.alternative');

      alts.forEach(function(element, index){
          element.textContent = q.alternatives[index];
      });
  },

  checkAnswer: function(userSelected) {
      if(this.currQuestion.correctAnswer == userSelected) {
          // correct
          console.log('correct');
      }
      else {
          // not correct
          console.log('wrong');
      }
  }
};

app.start();
```

Multiple Questions

script.js

Found in Project/Project 06 - Multiple Questions

The JavaScript code makes a quiz app that displays and cycles through an array of multiple-choice questions. When an answer is selected, it checks if it's correct, gives feedback, then moves to the next question. The app loops back to the first question if the end of the question array is reached.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];

let app = {
  start: function() {

    this.currPosition = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // show current question
    this.showQuestion(questions[this.currPosition]);
  },

  showQuestion: function(q) {

    // show question title
```



```
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.textContent = q.alternatives[index];
    });
},

checkAnswer: function(userSelected) {

    let currQuestion = questions[this.currPosition];

    if(currQuestion.correctAnswer == userSelected) {
        // correct
        console.log('correct');
    }
    else {
        // not correct
        console.log('wrong');
    }

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
},

increasePosition: function() {
    this.currPosition++;

    if(this.currPosition == questions.length) {
        this.currPosition = 0;
    }
}

};

app.start();
```

Showing the Score

script.js

Found in Project/Project 07 - Showing the Score

This JavaScript code builds a quiz app with a set of multiple-choice questions. It allows users to select an answer, which is checked for correctness, provides feedback, updates the score accordingly, and moves on to the next question. If the last question is reached, it loops back to the first one. It also keeps track of and displays the user's score.



```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];
```

```
let app = {
  start: function() {

    this.currPosition = 0;
    this.score = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // refresh stats
    this.updateStats();

    // show first question
    this.showQuestion(questions[this.currPosition]);
  },

  showQuestion: function(q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
```



```
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.textContent = q.alternatives[index];
    });
},

checkAnswer: function(userSelected) {

    let currQuestion = questions[this.currPosition];

    if(currQuestion.correctAnswer == userSelected) {
        // correct
        console.log('correct');
        this.score++;
    }
    else {
        // not correct
        console.log('wrong');
    }

    // refresh stats
    this.updateStats();

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
},

increasePosition: function() {
    this.currPosition++;

    if(this.currPosition == questions.length) {
        this.currPosition = 0;
    }
},

updateStats: function() {
    let scoreDiv = document.getElementById('score');
    scoreDiv.textContent = `Your score: ${this.score}`;
},

};

app.start();
```

Showing the Correct Answer

script.js



Found in Project/Project 08 - Showing the Correct Answer

This JavaScript code constructs a quiz app which cycles through a list of multiple-choice questions. It tracks and displays the user's score, gives them feedback on whether their selected answer is correct or not, and shows the correct answer if they got it wrong. Lastly, it advances to the next question once an answer is selected.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];

let app = {
  start: function() {

    this.currPosition = 0;
    this.score = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
        this.checkAnswer(index);
      });
    });

    // refresh stats
    this.updateStats();

    // show first question
    this.showQuestion(questions[this.currPosition]);
  },
```



```
showQuestion: function(q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.textContent = q.alternatives[index];
    });
},

checkAnswer: function(userSelected) {

    let currQuestion = questions[this.currPosition];

    if(currQuestion.correctAnswer == userSelected) {
        // correct
        console.log('correct');
        this.score++;
        this.showResult(true);
    }
    else {
        // not correct
        console.log('wrong');
        this.showResult(false);
    }

    // refresh stats
    this.updateStats();

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
},

increasePosition: function() {
    this.currPosition++;

    if(this.currPosition == questions.length) {
        this.currPosition = 0;
    }
},

updateStats: function() {
    let scoreDiv = document.getElementById('score');
    scoreDiv.textContent = `Your score: ${this.score}`;
},

showResult: function(isCorrect) {
    let resultDiv = document.getElementById('result');
```



```
    let result = '';

    // checks
    if(isCorrect) {
        result = 'Correct Answer!';
    }
    else {
        // get the current question
        let currQuestion = questions[this.currPosition];

        // get correct answer (index)
        let correctAnswIndex = currQuestion.correctAnswer;

        // get correct answer (text)
        let correctAnswText = currQuestion.alternatives[correctAnswIndex];

        result = `Wrong! Correct answer: ${correctAnswText}`;
    }

    resultDiv.textContent = result;
}

};

app.start();
```

Styling the App

index.html

Found in Project/Project 09 - Styling the App

This HTML code creates a webpage with a title, a list with four choices for a quiz, two divs to display results and scores, and a wrapper for styling. It also links an external CSS file 'style.css' for styling and an external JavaScript file 'script.js' for behavior.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Learn JS at Zenva</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <div id="wrapper">
      <div id="title"></div>
      <ul>
        <li class="alternative"></li>
        <li class="alternative"></li>
        <li class="alternative"></li>
        <li class="alternative"></li>
      </ul>
    </div>
  </body>
</html>
```



```
        <div id="result"></div>
        <div id="score"></div>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

script.js

Found in Project/Project 09 - Styling the App

The JavaScript code forms a quiz app which navigates through a multiple-choice question array. It keeps track of the user's choices, provides immediate feedback and displays the correct answer when a mistake is made, updates the user's score, and moves to the next question. The application also loops back to the first question after the last one.

```
let questions = [
  {
    title: 'gato',
    alternatives: ['dog', 'cat', 'bird', 'fish'],
    correctAnswer: 1
  },
  {
    title: 'ave',
    alternatives: ['mouse', 'hamster', 'lizard', 'bird'],
    correctAnswer: 3
  },
  {
    title: 'rata',
    alternatives: ['cat', 'fish', 'rat', 'shark'],
    correctAnswer: 2
  },
  {
    title: 'mosca',
    alternatives: ['fly', 'puma', 'fish', 'dog'],
    correctAnswer: 0
  }
];
```

```
let app = {
  start: function() {

    this.currPosition = 0;
    this.score = 0;

    // get alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach((element, index) => {

      element.addEventListener('click', () => {
        // check correct answer
```



```
        this.checkAnswer(index);
    });
});

// refresh stats
this.updateStats();

// show first question
this.showQuestion(questions[this.currPosition]);
},

showQuestion: function(q) {

    // show question title
    let titleDiv = document.getElementById('title');
    titleDiv.textContent = q.title;

    // show alternatives
    let alts = document.querySelectorAll('.alternative');

    alts.forEach(function(element, index){
        element.textContent = q.alternatives[index];
    });
},

checkAnswer: function(userSelected) {

    let currQuestion = questions[this.currPosition];

    if(currQuestion.correctAnswer == userSelected) {
        // correct
        console.log('correct');
        this.score++;
        this.showResult(true);
    }
    else {
        // not correct
        console.log('wrong');
        this.showResult(false);
    }

    // refresh stats
    this.updateStats();

    // increase position
    this.increasePosition();

    // show next question
    this.showQuestion(questions[this.currPosition]);
},

increasePosition: function() {
    this.currPosition++;

    if(this.currPosition == questions.length) {
```



```
        this.currPosition = 0;
    }
},

updateStats: function() {
    let scoreDiv = document.getElementById('score');
    scoreDiv.textContent = `Your score: ${this.score}`;
},

showResult: function(isCorrect) {
    let resultDiv = document.getElementById('result');
    let result = '';

    // checks
    if(isCorrect) {
        result = 'Correct Answer!';
    }
    else {
        // get the current question
        let currQuestion = questions[this.currPosition];

        // get correct answer (index)
        let correctAnswIndex = currQuestion.correctAnswer;

        // get correct answer (text)
        let correctAnswText = currQuestion.alternatives[correctAnswIndex];

        result = `Wrong! Correct answer: ${correctAnswText}`;
    }

    resultDiv.textContent = result;
}

};

app.start();
```

style.css

Found in Project/Project 09 - Styling the App

This CSS code sets styles for a webpage that includes a quiz app. It provides styling rules for the body, wrapper, title, alternative answers, and result and score fields of the app. These styles include dimensions, colors, fonts, and hover effects for better user interaction.

```
body {
    background: #1E90FF;
    font-family: arial;
    text-align: center;
}

#wrapper {
    margin-top: 40px;
```



```
    width: 100%;
    display: flex;
    flex-direction: column;
    align-items: center;
}

#wrapper div, #wrapper ul {
    width: 50%;
    max-width: 300px;
}

ul {
    padding: 0px;
}

#title {
    font-size: 20px;
    font-weight: bold;
    background: white;
    padding: 10px;
}

.alternative {
    list-style-type: none;
    margin-bottom: 4px;
    padding: 10px;
    background: #E6E6E6;
}

.alternative:hover {
    cursor: pointer;
    opacity: 0.8;
}

#result {
    font-weight: bold;
}

#score, #result {
    color: white;
}
```